

Constructing the Geometric Layer: Initialization, Parameter Blocks, and Optimization for Knowledge Graph Embeddings

Niklaus Parcell
Mont Ops Inc. / Independent Research
nik@mont-ops.com

May 4, 2026

Abstract

The linear-algebraic framework of Parcell [1] treats entity rows $\mathbf{e}_a$ and relation parameters θ_r as inputs to a scoring map $s(h, r, t)$. In deployment, those dense parameters are rarely provided *ab initio*; they must be *constructed* by allocation, optional structured initialization, and (most often) iterative optimization on link-prediction or ranking objectives coupled with negative sampling. This companion note formalizes the **parameter bundle** $(\mathbf{E}, \Theta)$, standard initialization choices, a generic stochastic training loop, and complexity at the level of scoring forward passes—complementing the structural (sparse) layer developed in the unified note.

1 Relationship to the unified framework

The unified note (henceforth UNIFIED) separates a sparse structural layer derived from asserted triples $\mathcal{T}$ from a geometric layer of embeddings and scores. UNIFIED is largely agnostic to *how* $\mathbf{E} \in \mathbb{R}^{N \times d}$ and relation parameters Θ arise. This note closes that gap: $\mathbf{E}$ and Θ are *outputs* of a construction pipeline (allocate $\rightarrow$ initialize $\rightarrow$ optionally train $\rightarrow$ persist), not prerequisites handed in by an oracle.

2 Structured parameter bundle

2.1 Entity embedding table

Definition 1 (Entity table). *The entity embedding matrix is $\mathbf{E} \in \mathbb{R}^{N \times d}$, whose a -th row $\mathbf{e}_a^\top$ is the embedding of entity $a \in [N]$. Storage is typically row-major dense layout in numerical libraries (each row contiguous).*

2.2 Relation parameter blocks

Definition 2 (Relation parameterization). *A relation block for type $r \in [L]$ is an ordered collection of tensors θ_r (vectors, matrices, or higher-order blocks) whose shapes are fixed by the scoring architecture. The full relation bundle is $\Theta = (\theta_1, \dots, \theta_L)$.*

Custom structure. Implementation-wise, Θ need not flatten into a single matrix; it may be a struct of arrays (e.g., one dense $\mathbf{W}_r \in \mathbb{R}^{d \times d}$ per relation for DistMult-style models, or lower-rank factors). What matters algorithmically is a deterministic map

$$(\mathbf{E}, \Theta) \mapsto s(h, r, t; \mathbf{E}, \Theta)$$

differentiable or subdifferentiable wherever optimization demands.

Remark 1 (Alignment with UNIFIED). UNIFIED *writes scores abstractly; this note treats $(\mathbf{E}, \Theta)$ as the mutable state vectorized logically (if not physically) for optimizers. Sparse operators $\mathbf{L}_k$ from $\mathcal{T}$ act on current node features (often initial rows of $\mathbf{E}$ or derived $\mathbf{H}$); training may alternate graph propagation and score-based losses.*

3 Allocation and initialization

3.1 Allocation

Given (N, L, d) and the per-relation shapes implied by the model class, an implementation:

1. allocates $\mathbf{E}$ with Nd scalars;
2. allocates each θ_r per schema;
3. optionally reserves optimizer state (momentum, second moments) of matching shapes.

3.2 Initialization strategies

Common choices (not mutually exclusive across ablations):

- **Random isotropic / fan schemes:** entries i.i.d. or scaled by layer width to control gradient magnitude at start.
- **Small-norm:** enforce $\|e_a\|$ bounds initially for translation models.
- **Deterministic prototype:** embed entity a from hashed metadata or degree statistics (weak prior, debugging).
- **Warm start:** load $(\mathbf{E}, \Theta)$ from checkpoint or external encoder, then continue training.
- **Non-learned placeholder:** $\mathbf{E}$ left fixed (e.g., zeros or one-hot padded); useful only for structural software tests.

Remark 2. *No initialization implies factual correctness; only optimization (or external grounding) aligns $\mathbf{E}$ with $\mathcal{T}$ and downstream tasks.*

4 Negative sampling and training batches

4.1 Positive minibatch

Fix batch size B . A minibatch $\mathcal{B}^+ = \{(h_b, r_b, t_b)\}_{b=1}^B$ samples asserted triples (with replacement or without, from empirical distribution over $\mathcal{T}$).

4.2 Corrupted negatives

For each positive (h, r, t) , generate one or more negatives by replacing t (or h) with an index drawn from a distribution q over $[N]$ (uniform, frequency-based, or adversarial). Denote negatives (h', r', t') .

4.3 Objectives (sketch)

Margin ranking.

$$\mathcal{L}_{\text{margin}} = \sum_b \max(0, \gamma - s(h_b, r_b, t_b) + s(h_b^{\text{neg}}, r_b, t_b^{\text{neg}})). \quad (1)$$

Binary logistic / softplus. Scores against sigmoid of $s_{\text{pos}} - s_{\text{neg}}$.

Multi-class / sampled softmax. Treat tail prediction among a candidate set including the true tail and K negatives.

5 Generic construction algorithm

Definition 3 (Construction pipeline). *A construction pipeline is a tuple (Init, Train?, Save) where Init produces $(\mathbf{E}^{(0)}, \Theta^{(0)})$, Train? optionally maps T steps to $(\mathbf{E}^{(T)}, \Theta^{(T)})$, and Save serializes tensors.*

Training loop (conceptual). For step $t = 0, \dots, T - 1$:

1. Sample minibatch $\mathcal{B}^+$ from $\mathcal{T}$; build negatives.
2. Evaluate forward scores s for positives and negatives using $(\mathbf{E}^{(t)}, \Theta^{(t)})$.
3. Compute loss $\mathcal{L}^{(t)}$ (optionally + regularizers on $\|\mathbf{E}\|_F$ or relation blocks).
4. Update parameters via a chosen optimizer (SGD, Adam, ...), respecting numeric dtype `f32/f64`.

Proposition 1 (Per-step dominant cost (forward)). *Suppose scoring each triple requires $\Theta(d)$ or $\Theta(d^2)$ work for bilinear models with $d \times d$ relation matrices, and batch size is B with K negatives per positive. Then one minibatch forward through scores is $\Theta(B(1 + K) \cdot C(d))$ where $C(d)$ is the per-triple asymptotic score cost. Sparse propagations $\mathbf{LX}$ incur additional $O(Zd)$ cost per application as in UNIFIED.*

Proof sketch. Count scalar operations in s for one triple under the chosen model; multiply by the number of scored triples in the batch. $\square$

6 Persistence and versioning

Serialize (N, L, d) , vocabulary maps, $\mathbf{E}$, flattened or structured Θ , optimizer metadata, and training configuration. Checksums or semantic versioning protect consumer code from silent shape mismatch.

7 Discussion

This note deliberately omits dataset-specific hyperparameter recipes and autodifferentiation plumbing; it specifies *what* must exist for $\mathbf{E}$ to be “created” in software: allocation, initialization, optional iterative fitting driven by $(\mathcal{T}, s, \mathcal{L})$, and persistence. Multimodal alignment losses add terms coupling rows of $\mathbf{E}$ to projected encoder outputs; structurally they extend $\mathcal{L}$ without changing the allocation story.

References

- [1] N. Parcell. *Sparse Knowledge Graphs, Joint Embeddings, and Multimodal Alignment: A Linear-Algebraic Framework for Structured World Models*. Companion technical note `kg_unified_multimodal.tex`, 2026.
- [2] A. Bordes, N. Usunier, A. García-Durán, J. Weston, O. Yakhnenko. Translating embeddings for modeling multi-relational data. In *NIPS*, 2013.
- [3] G. Niu et al. Knowledge graph embeddings: a comprehensive survey on capturing relation properties. *arXiv preprint* 2410.14733, 2025.